Amazon India Analysis Report

Problem-1.1

Amazon India wants to simplify its financial reports by standardizing payment values. Round the average payment values to integer (no decimal) for each payment type and display the results sorted in ascending order.

Output: payment_type, rounded_avg_payment

Approach:

1. **Identify Relevant Data:**
    - Focus on the payments table.
    - Use payment_type (e.g., credit card, voucher) and payment_value.
2. **Calculate Average Payment:**
    - Use AVG(payment_value) to compute the average for each payment type.
3. **Round the Average Payment:**
    - Apply ROUND(..., 0) to remove decimals.
4. **Group Data by Payment Type:**
    - Use GROUP BY payment_type to segment results.
5. **Sort the Results:**
    - Use ORDER BY rounded_avg_payment to display from lowest to highest.

Query:

sql

CopyEdit

```
SELECT payment_type, ROUND(AVG(payment_value), 0) AS rounded_avg_payment

FROM Payments

GROUP BY payment_type

ORDER BY rounded_avg_payment;
```

Output:

	payment_type character varying	rounded_avg_payment numeric
1	not_defined	0
2	voucher	66
3	debit_card	143
4	boleto	145
5	credit_card	163

Recommendations:

- Promote high-value payment types (credit_card, boleto) through loyalty programs.
- Offer incentives for underutilized methods (e.g., debit_card, voucher).

Problem-1.2

Amazon India wants to refine its payment strategy by analyzing the distribution of orders by payment type. Calculate the percentage of total orders for each payment type, rounded to one decimal place, and display them in descending order.

Output: payment_type, percentage_orders

Approach:

1. **Identify Relevant Data:**
    - Use the payments table with order_id and payment_type.
2. **Calculate Total Orders by Type:**
    - Use COUNT(DISTINCT order_id) to get unique orders.
3. **Compute Percentage of Orders:**
    - Divide count per type by total orders, multiply by 100.
4. **Round Percentage Values:**
    - Use ROUND(..., 1) for readability.
5. **Sort by Frequency:**
    - Use ORDER BY percentage_orders DESC.

Query:

sql

CopyEdit

SELECT payment_type,

ROUND(COUNT(payment_type) * 100.0 / (SELECT COUNT(payment_type) FROM Payments), 1) AS percentage_orders

FROM Payments

GROUP BY payment_type

ORDER BY percentage_orders DESC;

Output:

	payment_type character varying	percentage_orders numeric
1	credit_card	73.9
2	boleto	19.0
3	voucher	5.6
4	debit_card	1.5
5	not_defined	0.0

Recommendations:

- Encourage top-performing payment methods with cashback and promotions.
- Improve UX and incentives for low-performing payment methods.

Problem-1.3

Amazon India seeks to create targeted promotions for products priced between 100 and 500 BRL that contain the word 'Smart' in their name. Display these products, sorted by price in descending order.

Output: product_id, price

Approach:

1. **Identify Relevant Data:**
    - Use the product and order_items tables.
2. **Join Tables on Product ID:**
    - Perform INNER JOIN on product_id.
3. **Filter Products:**
    - Include only items with price between 100 and 500.
    - Include items where category name contains 'smart' (case-insensitive).

4. **Sort by Price:**
    - Use ORDER BY price DESC.

Query:

sql

CopyEdit

```
SELECT p.product_id, o.price, p.product_category_name
FROM product p
JOIN order_items o ON p.product_id = o.product_id
WHERE o.price BETWEEN 100 AND 500
AND LOWER(p.product_category_name) LIKE '%smart%'
ORDER BY o.price DESC;
```

Output:

Showing rows: 1 to 34

	product_id character varying	price numeric (10,2)	product_category_name character varying
1	dbd55362ec13c706503b1c71a5068...	102.00	smart
2	3168b2696b15ca440b92afa9e011a0...	109.90	smart
3	0e52955ca8143bd179b311cc454a6...	335.00	smart
4	dbd55362ec13c706503b1c71a5068...	102.00	smart
5	49ef750dc5bf23e3788d4f614bc6db...	198.00	smart
6	dbd55362ec13c706503b1c71a5068...	102.00	smart
7	7debe59b10825e89c1cbcc8b190c8...	349.99	smart
8	dbd55362ec13c706503b1c71a5068...	102.00	smart

Total rows: 34 | Query complete 00:00:00.295

Recommendations:

- Create mid-range "Smart" product bundles or flash sales.
- Position smart products with personalized ads for tech-savvy buyers.

Problem-1.4

Amazon India wants to identify seasonal sales patterns. Determine the top 3 months with the highest total sales value, rounded to the nearest integer.

Output: month, total_sales

Approach:

1. **Identify Relevant Data:**
    - Use the orders and order_items tables.
    - Extract order_purchase_timestamp for dates and price for sales.
2. **Join Orders with Order Items:**
    - Match on order_id.
3. **Extract Month & Year:**
    - Use EXTRACT(MONTH) and EXTRACT(YEAR) functions.
4. **Calculate Monthly Sales:**
    - Use SUM(price) to get total revenue by month.
5. **Sort & Limit:**
    - Sort by total_sales DESC.
    - Use LIMIT 3 to get top 3 months.

Query:

sql

CopyEdit

```
SELECT EXTRACT(YEAR FROM o1.order_purchase_timestamp) AS year,
    EXTRACT(MONTH FROM o1.order_purchase_timestamp) AS month,
    SUM(o2.price) AS total_sales
FROM orders o1
JOIN order_items o2 ON o1.order_id = o2.order_id
GROUP BY year, month
ORDER BY total_sales DESC
LIMIT 3;
```

Output:

	year numeric	month numeric	total_sales numeric
1	2017	11	1010271.37
2	2018	4	996647.75
3	2018	5	996517.68

Recommendations:

- Boost inventory in top-selling months.

- Launch pre-season campaigns to build sales momentum.

Problem-1.5

Amazon India is interested in product categories with significant price variations. Identify categories where the difference between maximum and minimum product prices is greater than 500 BRL.

Output: product_category_name, price_difference

Approach:

1. **Identify Relevant Data:**
    - Use product and order_items.
2. **Join Tables:**
    - Join on product_id.
3. **Compute Price Difference:**
    - Use MAX(price) - MIN(price).
4. **Filter for Significant Variation:**
    - Use HAVING clause to find categories where the difference > 500.

Query:

sql

CopyEdit

```
SELECT p.product_category_name,
    MAX(o.price) - MIN(o.price) AS price_difference
FROM product p
JOIN order_items o ON p.product_id = o.product_id
GROUP BY p.product_category_name
HAVING MAX(o.price) - MIN(o.price) > 500;
```

Output:

	product_category_name character varying	price_difference numeric
1	climatizacao	1588.10
2	livros_importados	730.01
3	[null]	3977.00
4	ferramentas_jardim	3923.65
5	dvds_blu_ray	1411.10
6	cine_foto	867.19
7	beleza_saude	3122.80
8	livros_interesse_geral	893.90

Total rows: 57 Query complete 00:00:00.341

Recommendations:

- Consider price-based filtering for customer-friendly sorting.
- Offer bundles or tiered discounts in categories with wide pricing gaps.

Problem-1.6

Amazon India wants to find which payment types have the most consistent transaction amounts. Identify payment types with the least variance, sorted by smallest standard deviation.

Output: payment_type, std_deviation

Approach:

1. **Use Payments Table:**
   - Analyze payment_type and payment_value.
2. **Calculate Standard Deviation:**
   - Use STDDEV(payment_value) to measure variability.
3. **Group & Sort:**
   - GROUP BY payment_type.
   - ORDER BY std_deviation ASC.

Query:

sql

CopyEdit

```
SELECT payment_type,
    STDDEV(payment_value) AS std_deviation
FROM payments
```

GROUP BY payment_type

ORDER BY std_deviation;

Output:

	payment_type character varying	std_deviation numeric
1	not_defined	0
2	voucher	115.519185430458
3	boleto	213.581061475527
4	credit_card	222.119310741208
5	debit_card	245.793401040647

Recommendations:

- Promote stable methods (e.g., vouchers) for predictability.
- Use more dynamic ones (e.g., credit) for flexible promotions.

Problem-1.7

Amazon India wants to identify products with incomplete names. Retrieve products where the category name is missing or contains only a single character.

Output: product_id, product_category_name

Approach:

1. **Analyze Product Table:**
   - Use product_id and product_category_name.
2. **Filter for Incomplete Entries:**
   - Use IS NULL to find missing data.
   - Use LENGTH(...) = 1 to catch one-character categories.

Query:

sql

CopyEdit

```
SELECT product_id, product_category_name
FROM product
WHERE LENGTH(TRIM(product_category_name)) = 1
  OR product_category_name IS NULL;
```

Output:

	product_id [PK] character varying	product_category_name character varying
1	a41e356c76fab66334f36de622ecbd3a	[null]
2	d8dee61c2034d6d075997acef1870e...	[null]
3	56139431d72cd51f19eb9f7dae4d1617	[null]
4	46b48281eb6d663ced748f324108c7...	[null]
5	5fb61f482620cb672f5e586bb132eae9	[null]
6	e10758160da97891c2fdcbc35f0f031d	[null]
7	39e3b9b12cd0bf8ee681bbc1c130feb5	[null]
8	794de06c32a626a5692ff50e4985d36f	[null]

Succes

Total rows: 614 | Query complete 00:00:00.227

Recommendations:

- Add validation in product upload pipeline.
- Regular audits can catch and fix missing data.

Problem-2.1

Amazon India wants to understand which payment types are most popular across different order value segments. Segment order values into:

- Low: < 200 BRL
- Medium: 200 to 1000 BRL
- High: > 1000 BRL
  Then calculate the count of each payment type in these ranges.

Output: order_value_segment, payment_type, count

Approach:

1. **Identify Relevant Data:**
   - Use the payments table containing payment_value and payment_type.
2. **Classify Orders by Value:**
   - Use CASE to label each payment as low, medium, or high.
3. **Count Payment Types by Segment:**
   - Use COUNT(payment_type).
4. **Group and Sort:**

- GROUP BY both segment and type.
- ORDER BY count DESC.

Query:

sql

CopyEdit

```
SELECT CASE
        WHEN payment_value < 200 THEN 'low'
        WHEN payment_value BETWEEN 200 AND 1000 THEN 'medium'
        ELSE 'high' END AS order_value_segment,
    payment_type,
    COUNT(payment_type) AS count
FROM payments
GROUP BY order_value_segment, payment_type
ORDER BY count DESC;
```

Output:

	order_value_segment text	payment_type character varying	count bigint
1	low	credit_card	60548
2	low	boleto	16444
3	medium	credit_card	15303
4	low	voucher	5476
5	medium	boleto	3162
6	low	debit_card	1287
7	high	credit_card	944
8	medium	voucher	286

Total rows: 13 | Query complete 00:00:00.156

Recommendations:

- Credit cards are popular in low/medium ranges — promote instant discounts.
- Increase voucher usage via cashback in low segments.

Problem-2.2

Amazon India wants to analyze price ranges and averages for each product category.

Output: product_category_name, min_price, max_price, avg_price

Approach:

1. **Identify Relevant Tables:**
   - Use product and order_items.
2. **Join Data on Product ID:**
   - Join product_id to link categories and prices.
3. **Calculate Price Metrics:**
   - Use MIN, MAX, and AVG, rounded to 2 decimal places.
4. **Group and Sort:**
   - Group by category.
   - Sort by avg_price DESC.

Query:

sql

CopyEdit

```
SELECT p.product_category_name,
    MIN(o.price) AS min_price,
    MAX(o.price) AS max_price,
    ROUND(AVG(o.price), 2) AS avg_price
FROM product p
JOIN order_items o ON o.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY avg_price DESC;
```

Output:

	product_category_name character varying	min_price numeric	max_price numeric	avg_price numeric
1	pcs	34.50	6729.00	1098.34
2	portateis_casa_forno_e_cafe	10.19	2899.00	624.29
3	eletrodomesticos_2	13.90	2350.00	476.12
4	agro_industria_e_comercio	12.99	2990.00	341.66
5	instrumentos_musicais	4.90	4399.87	281.62
6	eletroportateis	6.50	4799.00	280.78
7	portateis_cozinha_e_preparadores_de_alimentos	17.42	1099.00	264.57
8	telefonia_fixa	6.00	1790.00	225.69

Total rows: 79 | Query complete 00:00:00.292

Recommendations:

- Focus promotions on high-avg-price categories.
- Identify low-avg-price segments for bundling or upselling.

Problem-2.3

Amazon India wants to identify customers who have placed multiple orders over time. Retrieve their unique IDs and total number of orders.

Output: customer_unique_id, total_orders

Approach:

1. **Identify Relevant Tables:**
    - Use customers (for customer_unique_id) and orders (for order_id, customer_id).
2. **Join Data:**
    - INNER JOIN on customer_id.
3. **Count Orders Per Customer:**
    - Use COUNT(o.order_id) to calculate total orders.
4. **Filter for Repeat Customers:**
    - Use HAVING COUNT(o.order_id) > 1.
5. **Sort Results:**
    - Sort by total_orders DESC.

Query:

sql

CopyEdit

```
SELECT c.customer_unique_id,
    COUNT(o.order_id) AS total_orders
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_unique_id
HAVING COUNT(o.order_id) > 1
ORDER BY total_orders DESC;
```

Output:

	customer_unique_id character varying	total_orders bigint
1	0fdc0d21e1983e8af4d399e17671f76d	16
2	1b6d29725255a77667a8c639eeb4ccc0	16
3	363f980585bf04c1a88fdb986011c52e	16
4	3d364a7768fae99678635c4370295d20	16
5	4034aa08d48695a538b7030910aae5...	16
6	417b909c0962b2610f1cfeb1c1478986	16
7	5bf4ea2d98005b960eea0dbf652ef4e7	16
8	5f94af52aef02c968a2e0f01f430864e	16

Total rows: 3140 Query complete 00:00:01.404

Recommendations:

- Target repeat customers for loyalty programs and exclusive rewards.
- Use these insights to build retention strategies.

Problem-2.4

Amazon India wants to categorize customers based on their purchase frequency into:

- 'New' → 1 order
- 'Returning' → 2 to 4 orders
- 'Loyal' → more than 4 orders

Output: customer_id, customer_type

Approach:

1. **Create Temporary Classification:**
   - Use a WITH clause (CTE) or a temporary table to define customer types based on order count.
2. **Classify Customers with CASE:**
   - Based on COUNT(order_id).
3. **Join with Customers Table:**
   - Combine with the customers table to enrich the data.

Query:

sql

```
WITH categorize_customers AS (
  SELECT customer_id,
```

```
      CASE
        WHEN COUNT(order_id) = 1 THEN 'New'
        WHEN COUNT(order_id) BETWEEN 2 AND 4 THEN 'Returning'
        ELSE 'Loyal'
      END AS customer_type
  FROM orders
  GROUP BY customer_id
)
SELECT c.customer_id, t.customer_type
FROM customers c
JOIN categorize_customers t ON c.customer_id = t.customer_id;
```

Output:

	customer_id [PK] character varying	customer_type text
1	00012a2ce6f8dcda20d059ce984917...	New
2	000161a058600d5901f007fab4c271...	New
3	0001fd6190edaaf884bcaf3d49edf079	New
4	0002414f95344307404f0ace7a26f1d5	New
5	000379cdec625522490c315e70c7a9...	New
6	0004164d20a9e969af783496f34086...	New
7	000419c5494106c306a97b56357480...	New
8	00046a560d407e99b969756e0b10f2...	New

Total rows: 98348 | Query complete 00:00:02.988

Recommendations:

- Offer welcome coupons to New users.
- Send retention deals to Returning users.
- Build loyalty perks for high-value Loyal customers.

Problem-2.5

Amazon India wants to know which product categories generate the most revenue.

Output: product_category_name, total_revenue

Approach:

1. **Join Products and Order Items:**

    - Use product_id to link category and price.
2. **Calculate Revenue:**
    - Use SUM(price) to calculate total revenue per category.
3. **Group and Sort:**
    - Group by product_category_name.
    - Sort by total_revenue DESC.
    - Use LIMIT 5 for top categories.

Query:

sql

CopyEdit

```
SELECT p.product_category_name, SUM(o.price) AS total_revenue
FROM product p
JOIN order_items o ON o.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY total_revenue DESC
LIMIT 5;
```

Output:

	product_category_name character varying	total_revenue numeric
1	beleza_saude	1257865.34
2	relogios_presentes	1203060.32
3	cama_mesa_banho	1032268.59
4	esporte_lazer	985881.10
5	informatica_acessorios	910605.07

Recommendations:

- Promote top categories using personalized marketing.
- Highlight these categories on homepage and during sales.

Problem-3.1

The **marketing team** wants to compare total sales between different **seasons**:

- Spring (March–May)

- Summer (June–August)
- Autumn (September–November)
- Winter (December–February)

Output: season, total_sales

Approach:

1. **Identify Relevant Data:**
   - Use orders and payments.
2. **Join Tables:**
   - Match order_id to connect payment values with order dates.
3. **Categorize Seasons:**
   - Use CASE with EXTRACT(MONTH) to assign seasons.
4. **Aggregate Sales:**
   - Use SUM(payment_value) grouped by season.

Query:

sql

CopyEdit

```
SELECT season, total_sales FROM (
  SELECT CASE
        WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp) IN (3,4,5) THEN 'Spring'
        WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp) IN (6,7,8) THEN 'Summer'
        WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp) IN (9,10,11) THEN 'Autumn'
        ELSE 'Winter' END AS season,
      SUM(p.payment_value) AS total_sales
  FROM orders o
  JOIN payments p ON p.order_id = o.order_id
  GROUP BY season
) AS seasonal_sales
ORDER BY total_sales DESC;
```

Output:

	seasons text	total_sales numeric
1	Spring	4216721.54
2	Winter	2905750.03
3	Autumn	2348812.51
4	Summaer	4120359.62

Recommendations:

- Push sales in peak seasons (Spring/Summer).
- Offer clearance or themed sales in low-demand seasons.

Problem-3.2

The **inventory team** wants to find products with **above-average sales volume**.

Output: product_id, total_quantity_sold

Approach:

1. **Use Order Items Table:**
   - Count order_item_id per product.
2. **Calculate Overall Average Sales:**
   - Use subquery to compute average quantity sold per product.
3. **Filter Products:**
   - Use HAVING to select only products above average.

Query:

sql

CopyEdit

```
SELECT product_id, COUNT(order_item_id) AS total_quantity_sold
FROM order_items
GROUP BY product_id
HAVING COUNT(order_item_id) > (
  SELECT AVG(product_count) FROM (
    SELECT COUNT(order_item_id) AS product_count
    FROM order_items
    GROUP BY product_id
```

) AS avg_sales

)

ORDER BY total_quantity_sold DESC;

Output:

	product_id character varying	total_quantity_sold bigint
1	3d5837f86205fe83f03fb5f7e4d5b9cf	11
2	afeeea6271148ee1bb15173b8187c431	53
3	434487f82b5c35646bd8155cf1946179	4
4	e5063ce7fff1cf7cd528dc4c1e7dcba8	4
5	b25a0f93e25104798df2d1664495d157	4
6	6639a238ead6779d6ef0b3eea56f9f86	4
7	dceb3f67aef3484498a7caa4ba50f484	6
8	3c4e3782469a0f1ac459dc6c47ebef31	4

Total rows: 6366 Query complete 00:00:00.426

Recommendations:

- Ensure stock for fast-moving products.
- Consider discounts for slow-moving items to clear inventory.

Problem-3.3

The **finance team** wants to analyze monthly revenue trends for the year **2018**.

Output: month, total_revenue

Approach:

1. **Filter for 2018 Orders:**
   - Use EXTRACT(YEAR) = 2018.
2. **Join Orders and Order Items:**
   - Connect order_id to get prices per month.
3. **Extract Month & Aggregate:**
   - Use SUM(price) grouped by month.

Query:

sql

CopyEdit

SELECT EXTRACT(MONTH FROM o.order_purchase_timestamp) AS month,

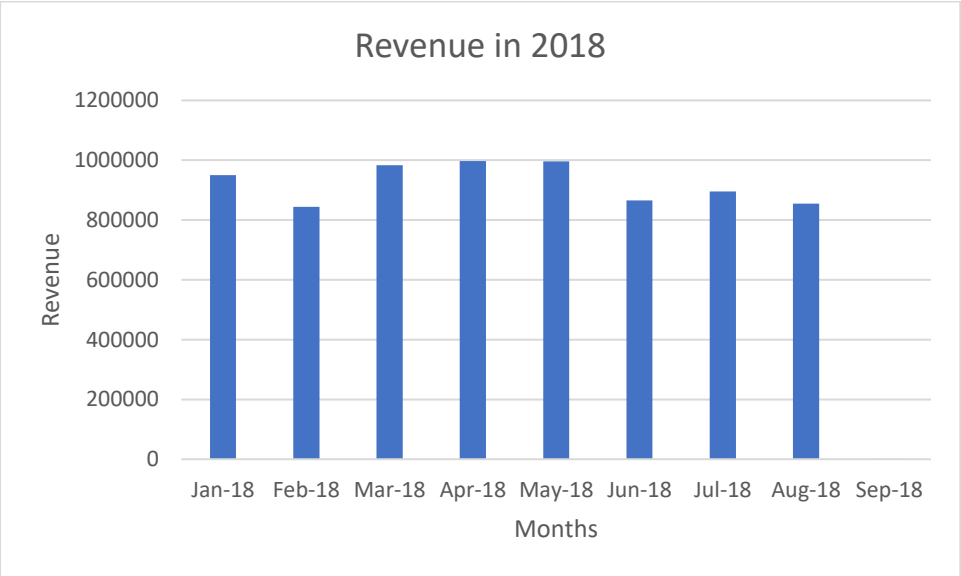
```
    SUM(oi.price) AS total_revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018
GROUP BY month
ORDER BY month;
```

Output:

	month numeric	sum numeric
1	1	950030.36
2	2	844178.71
3	3	983213.44
4	4	996647.75
5	5	996517.68
6	6	865124.31
7	7	895507.22
8	8	854686.33

Total rows: 9 Query complete 00:00:00.913

<u>Revenue Changes Across 2018</u>



Recommendations:

- Launch campaigns in months with historical revenue dips.
- Boost presence in peak months to maximize returns.

Problem-3.4

A **loyalty program** is being designed. Segment customers by purchase frequency:

- Occasional → 1–2 orders
- Regular → 3–5 orders
- Loyal → more than 5 orders

Output: customer_type, count

Approach:

1. **Use Orders Table:**
   - Count orders per customer_id.
2. **Classify Customer Type:**
   - Use CASE inside a CTE.
3. **Count Each Segment:**
   - Group by customer_type.

Query:

sql

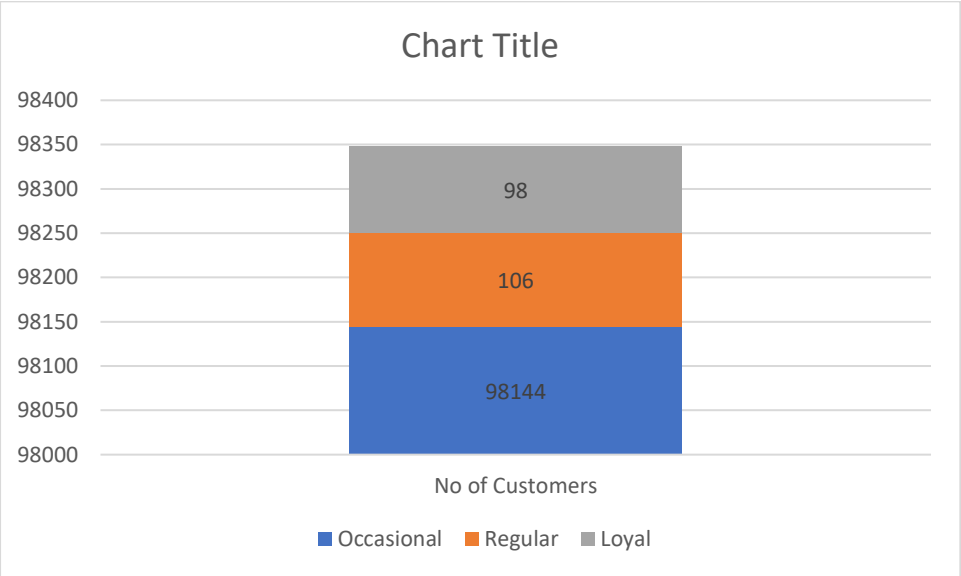
CopyEdit

```
WITH customertype AS (
  SELECT CASE
        WHEN COUNT(order_id) <= 2 THEN 'Occasional'
        WHEN COUNT(order_id) BETWEEN 3 AND 5 THEN 'Regular'
        ELSE 'Loyal'
       END AS customer_type,
       customer_id
  FROM orders
  GROUP BY customer_id
)
SELECT customer_type, COUNT(customer_id)
FROM customertype
GROUP BY customer_type;
```

Output:

	customer_type text	count bigint
1	Occasional	98144
2	Regular	106
3	Loyal	98

<u>Customer Segment</u>



Recommendations:

- Reward "Loyal" with exclusive perks.
- Turn "Occasional" into repeat buyers with first-purchase coupons.

Problem-3.5

Amazon wants to identify **high-value customers** based on average order value (AOV).

Output: customer_id, avg_order_value, customer_rank

Approach:

1. **Join Orders and Items:**
    - Use order_id to link data.
2. **Calculate AOV per Customer:**
    - Use AVG(oi.price) grouped by customer_id.
3. **Rank by Value:**
    - Use RANK() window function.

4. **Limit Results:**
    - Top 20 only.

Query:

sql

CopyEdit

```
SELECT o.customer_id,
    ROUND(AVG(oi.price), 0) AS avg_order_value,
    RANK() OVER (ORDER BY AVG(oi.price) DESC) AS customer_rank
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.customer_id
ORDER BY avg_order_value DESC
LIMIT 20;
```

Output:

Showing rows: 1 to 20

	customer_id character varying	avg_order_value numeric	customer_rank bigint
1	c6e2731c5b391845f6800c97401a43...	6735	1
2	f48d464a0baaea338cb25f816991ab1f	6729	2
3	3fd6777bbce08a352fddd04e4a7cc8f6	6499	3
4	df55c14d1476a9a3467f131269c2477f	4799	4
5	24bbf5fd2f2e1b359ee7de94defc4a15	4690	5
6	3d979689f636322c62418b6346b1c6...	4590	6
7	1afc82cd60e303ef09b4ef9837c9505c	4400	7
8	35a413c7ca3c69756cb75867d6311c...	4100	8

Total rows: 20 | Query complete 00:00:01.289

Recommendations:

- Design VIP offers for top customers.
- Gather feedback from them to improve services.

Problem-3.6

Amazon wants to track **monthly cumulative sales per product**.

Output: product_id, sale_month, total_sales

Approach:

1. **Aggregate Monthly Sales:**
   - Use DATE_TRUNC('month', ...) and SUM(price).
2. **Use Window Function:**
   - SUM(...) OVER (PARTITION BY product_id ORDER BY sale_month) for cumulative trend.
3. **Format Month:**
   - Use TO_CHAR(..., 'YYYY-MM').

Query:

sql

CopyEdit

```
WITH sales AS (
  SELECT oi.product_id,
      DATE_TRUNC('month', o.order_purchase_timestamp) AS sale_month,
      SUM(oi.price) AS monthly_sales
  FROM orders o
  JOIN order_items oi ON o.order_id = oi.order_id
  GROUP BY oi.product_id, sale_month
)
SELECT product_id,
     TO_CHAR(sale_month, 'YYYY-MM') AS sale_month,
     ROUND(SUM(monthly_sales) OVER (PARTITION BY product_id ORDER BY sale_month), 2) AS
total_sales
FROM sales
ORDER BY product_id, sale_month;
```

Output:

Showing rows: 1 to 1

	product_id character varying	sale_month text	total_sales numeric
1	00066f42aeeb9f3007548bb9d3f33c38	2018-05	101.65
2	00088930e925c41fd95ebfe695fd2655	2017-12	129.90
3	0009406fd7479715e4bef61dd91f2462	2017-12	229.00
4	000b8f95fcb9e0096488278317764d19	2018-08	117.80
5	000d9be29b5207b54e86aa1b1ac548...	2018-04	199.00
6	0011c512eb256aa0dbbb544d8dffcf6e	2017-12	52.00
7	00126f27c813603687e6ce486d909d01	2017-09	498.00
8	001795ec6f1b187d37335e1c4704762e	2017-10	38.90

Total rows: 62176 Query complete 00:00:01.823

Recommendations:

- Identify growth/lifecycle stages of products.
- Time promotions based on sales momentum.

Problem-3.7

Amazon wants to compute **monthly sales growth per payment type** for 2018.

Output: payment_type, sale_month, monthly_total, monthly_change

Approach:

1. **Aggregate Monthly Sales:**
   - Join orders, payments, order_items.
   - Group by payment_type and month.
2. **Calculate MoM Growth:**
   - Use LAG(...) and ROUND(...) to compute percentage change.
3. **Format Output:**
   - Format sale_month as YYYY-MM.

Query:

sql

CopyEdit

```
WITH monthly_sales AS (
  SELECT p.payment_type,
```

```
    DATE_TRUNC('month', o.order_purchase_timestamp) AS sale_month,
    SUM(oi.price) AS monthly_total
  FROM orders o
  JOIN payments p ON o.order_id = p.order_id
  JOIN order_items oi ON oi.order_id = o.order_id
  WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018
  GROUP BY p.payment_type, sale_month
),
sales_growth AS (
  SELECT payment_type,
    TO_CHAR(sale_month, 'YYYY-MM') AS sale_month,
    monthly_total,
    ROUND((monthly_total - LAG(monthly_total) OVER (PARTITION BY payment_type ORDER BY sale_month)) /
        NULLIF(LAG(monthly_total) OVER (PARTITION BY payment_type ORDER BY sale_month), 0) * 100, 2) AS monthly_change
  FROM monthly_sales
)
SELECT * FROM sales_growth
ORDER BY payment_type, sale_month;
```

Output:

	payment_type character varying	sale_month text	monthly_total numeric	monthly_change numeric
1	boleto	2018-01	170651.40	[null]
2	boleto	2018-02	153165.77	-10.25
3	boleto	2018-03	157807.38	3.03
4	boleto	2018-04	162940.61	3.25
5	boleto	2018-05	166571.60	2.23
6	boleto	2018-06	126379.90	-24.13
7	boleto	2018-07	162938.42	28.93
8	boleto	2018-08	118214.12	-27.45

Total rows: 33 | Query complete 00:00:00.880

Recommendations:

- Focus offers around methods with consistent or growing trends.

- Investigate dips and create incentives accordingly.